Q20. Email, URL and IP artifacts from an image

Original question: Analyze a forensic disk image to identify network-related artifacts, including available email addresses, URLs, and IP addresses. Examine the generated output files and document your findings.

Aim

Extract candidate email addresses, URLs and IP-related artifacts, validate their context and document traceable findings.

Requirements

Supplied image/segments, bulk_extractor, a hex viewer and Autopsy/TSK as needed for context. This extends Q12 by explicitly inspecting IP artifacts and validating address candidates. Reference: [R11].

Procedure

Step 1 - Prepare and run extraction

```
mkdir -p "$HOME/EHF_Exam/Q20"
cd "$HOME/EHF_Exam/Q20"
IMAGE='<DISK_IMAGE_PATH>'
sha256sum "$IMAGE" | tee image_before.sha256
bulk_extractor -h > bulk_help.txt
OUT="$PWD/bulk_q20_run1"
test ! -e "$OUT" && bulk_extractor -o "$OUT" "$IMAGE" \
  > bulk_run.log 2>&1
cat bulk_run.log
ls -l "$OUT"
```

Use a fresh output name if it exists. Confirm the installed email, URL and network/IP-related scanners in help and check completion/errors. The relevant network scanner is commonly net, not necessarily a scanner named ip; use installed help before enabling any nondefault scanner. File names and feature coverage depend on build and data. If Q12 already produced a verified complete output for this same image, it may be reused with the original run log rather than rescanning; record that reuse.

Step 2 - Inspect three feature categories

```
for NAME in email.txt url.txt ip.txt; do
  if [ -f "$OUT/$NAME" ]; then
    printf '\n--- %s ---\n' "$NAME"
    sed -n '1,60p' "$OUT/$NAME"
  else
    printf 'Not generated: %s\n' "$NAME"
  fi
done
for NAME in email_histogram.txt url_histogram.txt ip_histogram.txt; do
  if [ -f "$OUT/$NAME" ]; then
    sed -n '1,25p' "$OUT/$NAME"
  fi
done
```

Keep feature positions and context, not just unique address lists. IP features can come from recognized network structures; do not assume ip.txt contains every IP-looking string in the image, or that it always uses the same context layout.

Step 3 - Optional printable IPv4 candidate search

If the image is a verified raw byte image, this fallback identifies printable dotted-decimal candidates that may not appear in ip.txt. Do not run it directly on a compressed image container and treat container offsets as raw-disk offsets. Set RAW_IMAGE to a raw image or a verified lab-approved raw export.

```
RAW_IMAGE='<RAW_DISK_IMAGE_PATH>'
LC_ALL=C strings -a -t d "$RAW_IMAGE" \
  | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' \
  > ipv4_string_candidates.txt
sed -n '1,40p' ipv4_string_candidates.txt
```

`strings -t d` reports the decimal start offset of each printable string, not the exact start of the matched IP substring. This regex deliberately generates candidates: it can match 999.999.999.999, version strings or substrings, and misses binary/encoded/IPv6 addresses. Validate each selected literal offline with Python's ipaddress module:

```
python3 - '<CANDIDATE_IP_ADDRESS>' <<'PY'
import ipaddress
import sys
try:
    address = ipaddress.ip_address(sys.argv[1])
except ValueError as error:
    print('Invalid IP literal:', error)
else:
    print('Valid literal:', address, '| IP version:', address.version)
PY
```

This validates syntax only. For IPv6 candidates observed in extracted content, the same validator accepts IPv6 literals. Coverage is not comprehensive. Record private/loopback/link-local/documentation/public context carefully; a valid address does not prove communication.

Step 4 - Correlate context and retain an evidence trail

For each relevant email/URL/IP, save the feature line, position/path, source hash and tool version. For a simple raw byte offset, inspect bounded context offline:

```
OFFSET='<CONFIRMED_RAW_BYTE_OFFSET>'
xxd -s "$OFFSET" -l 256 "$RAW_IMAGE" > candidate_context.hex
cat candidate_context.hex
sha256sum -c image_before.sha256
```

Use the offset's actual numeric value. A bulk_extractor forensic path containing decompression/encoding components must be resolved with suitable tool support; do not give it directly to xxd. Correlate with file paths via Autopsy/TSK when possible, or mark source file unresolved. Never browse extracted URLs or contact extracted IPs merely to validate them.

Expected output - illustrative findings format

```
Type   | Candidate               | Position | Context
Email  | analyst@example.org     | [actual] | [actual source]
URL    | https://lab.example.org/ | [actual] | [actual source]
IP     | 192.0.2.25              | [actual] | [actual source]
```

These values are teaching examples, not forensic discoveries. Use only categories actually present and state scanner/coverage limits when absent.

Interpretation, observation and result

An address may come from documentation, caches, logs, embedded binaries or deleted content. A log with direction/time may support an event; an isolated string does not prove a connection, exfiltration or ownership. Histograms count extracted occurrences, not unique network sessions. Distinguish an observed string, syntactically valid IP and corroborated network event. The result is a supported inventory of all three requested categories where present, with source locations, contextual interpretation and limitations.

Common errors

- ip.txt missing/empty: inspect scanners and logs; use the raw-string candidate method only where appropriate and state its limits.
- Out-of-range address candidate: reject it as an IP literal while preserving the original observation if relevant.
- Offset shows unrelated data: confirm raw image versus container, decimal/hex units and recursive forensic paths.

Viva questions

- Does a valid IP string prove a connection? No; a relevant event record or packet evidence is needed.
- How does Q20 extend Q12? It adds IP artifacts and their syntax/context validation.
- Why not trust the IPv4 regex alone? It accepts invalid octets and unrelated number patterns.
- Are histogram counts network sessions? No; they are extraction occurrence counts.